

Lesson Task and Research Summary

Unity Event-Driven Programming and ShootingController.cs

Group: Happy Unity | Date: 5.14

1. Overview of the Class Task

The task for this lesson is to read a real Unity C# script like game programmers and explain how the script reacts to Unity events. Instead of writing a large new feature, the group focuses on understanding the event-driven structure of Unity code. The main idea is that Unity owns the game loop. Our script responds when Unity calls lifecycle and update event functions such as `OnEnable()`, `Update()`, and `OnDisable()`.

2. Main Topic

- Event-driven programming in Unity.
- Input handling and shooting logic in `ShootingController.cs`.

3. Script Analyzed

ShootingController.cs is the script we analyze. It controls firing behavior, reads an input action, checks whether the shooter is player controlled, controls the firing rate, spawns a projectile prefab, and optionally plays firing effects and sounds.

4. Main Concepts to Discuss

Concept	What we need to explain
InputAction	An input action represents a gameplay action such as Fire. It separates the action from the physical key, mouse button, or controller button used to trigger it.
Unity Event Functions	Functions such as <code>OnEnable()</code> , <code>Update()</code> , and <code>OnDisable()</code> are called automatically by Unity at specific moments in the component lifecycle.
MonoBehaviour Lifecycle	A MonoBehaviour script reacts to Unity lifecycle events instead of controlling the entire game loop manually.
Runtime Object Spawning	<code>Instantiate()</code> creates a new object while the game is running. In this script, it is used to spawn a projectile prefab.

5. Main Event Chain

The main event chain shows how a button input becomes a visible projectile in the game scene:

```
Update()
-> ProcessInput()
-> fireAction.ReadValue<float>()
-> Fire()
-> SpawnProjectile()
-> Instantiate(projectilePrefab)
-> A projectile appears in the scene
```

Explanation: Unity calls Update() every frame. Update() calls ProcessInput(), and ProcessInput() checks the current value of the fire input action. If the value is high enough, Fire() is called. Fire() then uses SpawnProjectile() to create the projectile prefab at runtime.

6. What We Will Explain in the Presentation

- How Unity automatically calls event functions instead of requiring us to write the game loop manually.
- How player input is checked every frame through Update() and ProcessInput().
- How InputAction works in the newer Unity Input System.
- How a projectile is dynamically created at runtime using Instantiate().
- Why Unity uses an event-driven architecture for gameplay systems.

7. Code Evidence We Will Use

The following code examples will be used as evidence. Each example connects a Unity event or helper method to the shooting system.

7.1 Enabling and disabling input

```
private void OnEnable()
{
    fireAction.Enable();
}

private void OnDisable()
{
    fireAction.Disable();
}
```

Meaning: When the component becomes active, the fire action is enabled so it can read input. When the component becomes inactive, the fire action is disabled so it stops reading input.

7.2 Checking input every frame

```
private void Update()
{
    ProcessInput();
}

private void ProcessInput()
{
    if (fireAction.ReadValue<float>() >= 1)
    {
        Fire();
    }
}
```

Meaning: Update() is called every frame. The script reads the current input value. For a button input, the value is usually 0 when not pressed and 1 when pressed. If the value is greater than or equal to 1, the script calls Fire().

7.3 Spawning a projectile

```
GameObject projectileGameObject = Instantiate(
    projectilePrefab,
    transform.position,
    transform.rotation,
    null
);
```

Meaning: Instantiate() creates a new copy of the projectile prefab at the shooter object's position and rotation. This is how the projectile appears in the scene during gameplay.

8. Research Summary

The research should focus on official Unity documentation first, then community explanations if needed. The most important research questions are:

- When does Unity call OnEnable(), Update(), and OnDisable()?
- Why does an InputAction need to be enabled before it can read input?
- What does ReadValue<float>() return for a button input?
- How does Instantiate() create a runtime object from a prefab?
- How does event-driven programming help organize gameplay systems?

9. Improvement Idea

Suggested improvement: Add a clearer prefab check in `SpawnProjectile()`. The current shooting logic should avoid failing silently if `projectilePrefab` is missing in the Inspector.

```
public void SpawnProjectile()
{
    if (projectilePrefab == null)
    {
        Debug.LogError("Projectile prefab is missing on " + gameObject.name);
        return;
    }

    GameObject projectileGameObject = Instantiate(
        projectilePrefab,
        transform.position,
        transform.rotation,
        null
    );
}
```

Why this helps: If the prefab is not assigned, the developer gets a clear error message. This improves debugging efficiency and helps prevent confusion when pressing the fire button does not create a projectile.

10. Reflection / Learning Outcomes

- We understand the Unity lifecycle better, especially how `OnEnable()`, `Update()`, and `OnDisable()` are called.
- We understand event-driven programming more clearly: Unity sends events, and our script responds.
- We understand how gameplay systems connect through an event chain, from input to method calls to visible game reactions.
- We understand how Unity manages player input through `InputAction` and runtime object generation through `Instantiate()`.
- We understand that helper functions such as `ProcessInput()`, `Fire()`, and `SpawnProjectile()` are not Unity event functions. They run because Unity event functions call them.

11. Final Takeaway

ShootingController.cs is a clear example of event-driven programming in Unity. Unity calls `Update()` every frame, the script reads the fire input, and the event chain leads to a projectile being instantiated in the game scene. This shows how Unity gameplay code is organized around engine events, input actions, helper functions, and runtime object spawning.